Name: Monishvaran V
Reg no:192424398

DSA0310 – Natural Language Processing

Assignment 2 (CO2): Model Answers – N-gram Language Model and POS Tagger for a Public Transport Enquiry Helpline

How to read this document. Your corpus (20,000 utterances) and your annotated data are your own, so measured figures cannot be given here. They are marked **[from your run]**. To keep every explanation checkable, all worked examples use one small **toy corpus of 10 helpline utterances** whose numbers were computed with the Python code in the Appendix. Replace the toy numbers with your own when you write up, using exactly the same method. Figures labelled *illustrative* are teaching values, not measurements.

Toy corpus (used throughout):

```
<s> two tickets to Vellore </s>
<s> two tickets to Madurai </s>
<s> one ticket to Vellore </s>
<s> book two tickets to Chennai </s>
<s> two tickets to Vellore please </s>
<s> one ticket to Chennai </s>
<s> book one ticket to Madurai </s>
<s> what is the fare to Vellore </s>
<s> what is the fare to Chennai </s>
<s> two tickets to Chennai </s>
```

Task 1 – Counting Words and Building the Vocabulary (3 marks)

Type, token, lemma, word form

Term	Definition	Toy corpus count
Token	Every running occurrence of a word	47 word tokens (67 with <s> and </s>)
Type	Each distinct token string	14 types (16 with <s> and </s>)
Word form	A distinct inflected surface form (here equal to types, as case is folded)	14
Lemma	Dictionary headword grouping inflections (ticket, tickets → ticket)	13

Report all four counts for your corpus **[from your run]**.

Decisions: fold case (Vellore = vellore, so one type, not two); keep station names as one token each; keep "6:30" as **one** token (the time is one unit of meaning); keep numerals as they are but optionally map to a class such as <NUM>; remove fillers (uh, um) or mark them as <DISF> and state which; add <s> at the start and </s> at the end of every utterance so that the first word and the end of the utterance get probabilities; remove punctuation because the transcripts have none.

Effect of tokenisation decisions

Decision	Vocabulary effect	Why
Case-sensitive vs case-folded	Case-sensitive keeps Vellore and vellore as 2 types; folded gives 1	Fewer types, denser counts, but proper-noun cue lost

Decision	Vocabulary effect	Why
"6:30" as one token vs three (6, :, 30)	One token: many rare time types (6:30, 6:45 …). Three tokens: only 3 types	Splitting shrinks the vocabulary but breaks the time into meaningless pieces for the n-gram

Measure the vocabulary size under both choices on your corpus **[from your run]**.

Zipf's law

Zipf's law: the frequency f of a word is inversely proportional to its rank r, $f \approx k / r$ (equivalently, a straight line of slope about −1 on a log-log plot of frequency against rank). A refinement (Mandelbrot) is $f = P(r + \rho)^{(-B)}$.

In the toy corpus, to (10) is rank 1, then two and tickets (5), Vellore and Chennai (4), one and ticket (3), then a long tail of words with counts 2 and 1. Plot log(rank) against log(frequency) for your corpus and fit the slope **[from your run]**.

Words that deviate most: (1) the preposition **to** is far more frequent than Zipf predicts, because every enquiry names a destination; (2) place names (Vellore, Chennai) rank much higher than ordinary nouns would, because the domain is about places; (3) function words such as the, of, a rank lower than in general English, because the utterances are short and telegraphic.

Unknown-word threshold (words occurring fewer than t times become <UNK>)

Threshold t	Toy vocabulary size (words)	Tokens sent to <UNK>	% of tokens
1	14	0	0%
2	13 (please removed)	1	2.1%
3	7	13	27.7%
5	3 (to, two, tickets)	27	57.4%

For your corpus report vocabulary size and % of tokens sent to <UNK> at t = 1, 2, 3, 5 **[from your run]**. Add 3 to the model vocabulary for <UNK>, <s>, </s>.

OOV rate

OOV rate = (development tokens whose type is not in the training vocabulary) ÷ (all development tokens) **[from your run]**. It limits everything that follows because an unseen word cannot receive a meaningful probability, a tag from the dictionary, or a slot in an extraction template. If 4% of tokens are OOV and most are place names, the system loses the information that the caller's request turns on, however good the smoothing.

Task 2 – Unsmoothed N-gram Models (4 marks)

Chain rule and the Markov assumption

Chain rule (exact): $P(w_1 \ldots w_n) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1 w_2) \cdot \ldots \cdot P(w_n|w_1 \ldots w_{n-1})$.

Bigram (first-order Markov) assumption: $P(w_k \mid w_1 \ldots w_{k-1}) \approx P(w_k \mid w_{k-1})$. What is assumed away: everything older than the single previous word. The caller's earlier words (for example the origin, when predicting the fare word) are ignored.

Trigram (second-order): $P(w_k \mid w_1 \ldots w_{k-1}) \approx P(w_k \mid w_{k-2}, w_{k-1})$. Only the two previous words are kept, so more context is used, but the number of parameters (V^3 possible trigrams) and the sparsity grow sharply.

Maximum likelihood estimators

```
Unigram: P(w)       = C(w) / N
Bigram:  P(w|a)     = C(a w) / C(a)
Trigram: P(w|a b)   = C(a b w) / C(a b)
```

Build all three from the training split only.

Hand-computed utterance probabilities (toy bigram model)

Utterance	Factors	Probability
two tickets to Vellore	P(two|<s>)=4/10 × P(tickets|two)=5/5 × P(to|tickets)=5/5 × P(Vellore|to)=4/10 × P(</s>|Vellore)=3/4	0.4×1×1×0.4×0.75 = 0.12
book one ticket to Madurai	2/10 × 1/2 × 3/3 × 3/3 × 2/10 × 2/2	0.2×0.5×1×1×0.2×1 = 0.02
what is the fare to Chennai	2/10 × 2/2 × 2/2 × 2/2 × 2/2 × 4/10 × 4/4	0.2×1×1×1×1×0.4×1 = 0.08
two tickets to Madurai please	0.4 × 1 × 1 × 0.2 × **P(please|Madurai)=0/2** × 1	**0**

The zero: "two tickets to Madurai please" is perfectly reasonable English, but the bigram **(Madurai, please)** never occurs in training, so its MLE is 0/2 = 0 and the whole product collapses to zero.

Sparsity

Toy corpus: 22 distinct bigrams observed out of 16 × 16 = 256 possible, i.e. **8.6%**; so 91.4% of possible bigrams are unseen. Report for your corpus: (a) proportion of possible bigrams and trigrams observed; (b) proportion of development-set bigrams and trigrams never seen in training **[from your run]**. The trigram figures will be much worse than the bigram figures.

Generation

Sample by repeatedly drawing the next word from the model's distribution (fixed seed 42) and stopping at </s>. Typical pattern of results (*illustrative*):

Model	Sample	Comment
Unigram	to two the tickets Vellore fare	Right words, no order; ungrammatical
Bigram	what is the fare to Madurai	Locally fluent, can drift, e.g. "two tickets to Vellore please to"
Trigram	book two tickets to Chennai	Close to copying training sentences

What order buys: more fluency and fewer impossible word pairs. **What it costs:** many more parameters, much more sparsity, and a tendency to reproduce training sentences, which is overfitting.

Task 3 – Smoothing and Discounting (5 marks)

Why zero is a modelling failure

A zero says the sentence is impossible, but it only means "not in my sample". With zero probability, one unseen bigram sets the product (and the perplexity) to 0 / infinity and an otherwise correct hypothesis can never be chosen. **Condition for a valid smoothing method:** for every context, the

probabilities of all possible next words must still be non-negative and sum to exactly 1, so mass given to unseen events must be taken from seen events.

Add-one and add-k (context = "to", V = 16, C(to) = 10)

Formulas: P_add-k(w|a) = (C(aw) + k) / (C(a) + kV); effective count $c* = (C(aw) + k) \cdot C(a) / (C(a) + kV)$.

Next word	Count	c* (k=1)	P (k=1)	c* (k=0.1)	P (k=0.1)
Vellore	4	1.923	0.1923	3.534	0.3534
Chennai	4	1.923	0.1923	3.534	0.3534
Madurai	2	1.154	0.1154	1.810	0.1810
please (unseen)	0	0.385	0.0385	0.086	0.0086
Each of the other 12 unseen words	0	0.385	0.0385	0.086	0.0086

MLE gave P(Vellore|to) = 0.4. After add-one it is 0.1923, so the seen words lose 1 − 0.5 = **0.5 of the probability mass** (total seen mass drops from 1.0 to 0.5) and the 13 unseen words share that 0.5. With k = 0.1 the seen words keep 0.888 and the unseen words get 0.112.

Why add-one is too severe for a large vocabulary

The unseen mass is (V − s) / (C + V), where s is the number of seen types. For a context seen C = 500 times with s = 100 distinct followers and V = 5,000 (*illustrative; use one from your corpus*): (5,000 − 100) / (500 + 5,000) = **89%** of the mass goes to the 4,900 unseen words, so the 500 real observations keep only about 11%. The effective count of a word seen 100 times drops to (101 × 500)/5,500 ≈ 9.2. Most of that mass is wasted on implausible words.

Good–Turing discounting

Method: let N_r be the number of bigram types seen exactly r times. Revised count $\mathbf{c* = (c + 1) \cdot N_{c+1} / N_c}$. Total probability of all unseen bigrams = $\mathbf{N_1 / N}$ (N = total bigram tokens); it is shared among the unseen types (V^2 − seen types).

Illustrative frequency-of-frequencies table for a larger corpus with N = 100,000 bigram tokens:

r	N_r	$c* = (r+1)N_{r+1}/N_r$	Discount c*/r	Note
1	10,000	2×3,000/10,000 = 0.60	0.60	big cut for singletons
2	3,000	3×1,500/3,000 = 1.50	0.75	
3	1,500	4×900/1,500 = 2.40	0.80	
4	900	5×600/900 = 3.33	0.83	
0 (unseen)	–	P_0 total = N_1/N = 10,000/100,000 = 0.10	–	10% of mass for unseen

High-frequency tail: for large r, N_r is small or zero ($N_{r+1} = 0$ gives $c* = 0$, which is wrong). So Good–Turing is applied only for $r \leq k$ (typically k = 5) and counts above k are left undiscounted (Katz), or the N_r values are first smoothed by a line fit in log-log space (Simple Good–Turing).

Toy-corpus warning: the toy corpus gives $N_1 = 4$, $N_2 = 9$, $N_3 = 3$, $N_4 = 4$, $N_5 = 2$, so $c*(1) = 4.5$, which is absurd (a singleton cannot be worth more than 1). It shows that Good–Turing needs smooth, plentiful N_r values, which is why it is fitted on the 20,000-utterance corpus and not on a sample this small.

Comparison and deployment choice

Measure development-set perplexity for MLE (infinite), add-one, add-k (tune k on the development set), and Good–Turing **[from your run]**. Also compare the estimate for one unseen bigram, for example P(please|to): add-one gives 0.0385, which is too high, because 13 unseen words get the same share; add-0.1 gives 0.0086; Good–Turing gives a much smaller, count-based value. **Expected ordering:** add-one worst, add-k better, Good–Turing better again. **Deploy** Good–Turing (inside Katz backoff) or interpolation, since pure add-k/add-one smoothing is poor for n-grams.

Task 4 – Backoff and Deleted Interpolation (4 marks)

Definitions

Backoff: use the trigram estimate if the trigram was seen; otherwise back off to the bigram, then to the unigram. **Interpolation:** always mix all orders, $P = \lambda_3 P(w|uv) + \lambda_2 P(w|v) + \lambda_1 P(w)$, with $\lambda_3 + \lambda_2 + \lambda_1 = 1$. **Difference:** backoff uses a shorter context only when the longer one has no count; interpolation always combines all orders, so even seen trigrams get some mass from lower orders.

Katz backoff and the weight α (toy bigram level, absolute discount d = 0.5 as a stand-in for Good–Turing)

```
P_katz(w|a) = P*(w|a)            if C(a w) > 0
            = α(a) · P(w)        otherwise
α(a) = β(a) / Σ_{w unseen after a} P(w),   β(a) = 1 − Σ_{w seen} P*(w|a)
```

Worked example, context "to" (C = 10; followers Vellore 4, Chennai 4, Madurai 2): discounted $P*$ = 3.5/10, 3.5/10, 1.5/10, which sum to 0.85, so $\beta = 0.15$. Unigram mass of words never seen after "to" = 1 − 10/67 = 0.8507. **α(to) = 0.15 / 0.8507 = 0.1763.** Then P_katz(please|to) = 0.1763 × (1/67) = **0.00263**.

Why α is essential: without α, the seen words already use 0.85 of the mass and the unseen words would add their full unigram mass of 0.8507, giving a total of about 1.70. With α the total is exactly 1.

Stupid backoff: S(w|a) = C(aw)/C(a) if seen, else 0.4 · P(w). For context "to": seen words give 0.4 + 0.4 + 0.2 = 1.0 and the unseen words add 0.4 × 0.8507 = 0.34, so the total is 1.34 ≠ 1. It is therefore a **score**, not a probability, because no discounting or α normalisation is done; it is cheap and works well on very large corpora but cannot be used for perplexity.

Simple and deleted interpolation

Simple linear interpolation (toy, $\lambda_2 = 0.7$ bigram, $\lambda_1 = 0.3$ unigram): P(please|Madurai) = 0.7 × 0 + 0.3 × (1/67) = **0.00448**; P(</s>|Madurai) = 0.7 × 1 + 0.3 × (10/67) = 0.7448.

Deleted interpolation: (1) split the training data into a main part and a held-out part; (2) count n-grams on the main part; (3) choose λ to maximise the likelihood of the held-out part using EM: for each held-out n-gram compute each component's responsibility $\lambda_i P_i / \Sigma \lambda_j p_j$, add them up, renormalise to get new λ; repeat to convergence (the function deleted_interp_lambda in the Appendix). The λ weights are never tuned on the test set.

λ depends on context count

A context seen only twice gives an unreliable higher-order estimate, so its λ should be small; a context seen 5,000 times gives a reliable one, so its λ should be large. Example of the effect (toy): for context "Madurai" (C = 2) with $\lambda_2 = 0.4$: P(please|Madurai) = 0.6 × 1/67 = 0.0090 (more weight on

the unigram), compared with $\lambda_2 = 0.7$ for a frequent context. Bands are normally formed by C(context): *illustrative* λ values (replace with your EM output **[from your run]**):

C(context) band	λ_3 (trigram)	λ_2 (bigram)	λ_1 (unigram)
1–5	0.15	0.45	0.40
6–50	0.45	0.40	0.15
>50	0.75	0.20	0.05

Hand trace of one utterance: "two tickets to Madurai please" (toy)

Step	MLE	Interpolation (λ=0.7)	Katz (d=0.5)	Add-one
P(two|<s>)	0.4	0.3024	0.35	0.1923
P(Madurai|to)	0.2	0.1490	0.15	0.1154
P(please|Madurai) (unseen)	**0**	**0.00448**	**0.00439**	0.0556
P(</s>|please)	1	0.7448	0.5	0.1176

Difference: for the unseen bigram, backoff goes to the unigram only because the bigram failed and scales it by α(Madurai) = 0.25/0.8507 = 0.2939, giving 0.00439; interpolation mixes the unigram in at weight 0.3 whether or not the bigram was seen, giving 0.00448. For the **seen** bigram (Madurai, </s>) Katz gives 0.75 (discounted MLE) while interpolation gives 0.7448 (MLE blended with the unigram). In a trigram model the same happens one level up: for an unseen trigram (to, Madurai, please) Katz falls to P(please|Madurai) scaled by α(to Madurai), and interpolation computes $\lambda_3 \cdot 0 + \lambda_2 \cdot P(\text{please}|\text{Madurai}) + \lambda_1 \cdot P(\text{please})$.

Whole-utterance probability: MLE = 0; interpolation = 8.08×10^{-5}; add-one = 1.18×10^{-5}. Compare backoff and interpolation by development perplexity on your corpus **[from your run]**.

Task 5 – Entropy, Cross-Entropy and Perplexity (4 marks)

Definitions and relation

- **Entropy** of a distribution: $H(p) = -\Sigma\, p(x) \log_2 p(x)$ bits, the average uncertainty (average number of yes/no questions needed to identify the outcome).
- **Cross-entropy** of model q on data with true distribution p: $H(p,q) = -\Sigma\, p(x) \log_2 q(x)$. For a test set of n words: $H = -(1/n)\, \Sigma \log_2 P_model(w_i \mid \text{history})$. It is always ≥ H(p); the closer, the better the model.
- **Perplexity** PP = 2^H, equivalently $PP = P(w_1 \ldots w_n)^{(-1/n)}$.

Perplexity 90 in plain words: on average the model is as uncertain about the next word as if it were choosing uniformly among 90 equally likely words (an average branching factor of 90). Lower is better.

Entropy of three distributions (toy model)

Distribution	Probabilities	H (bits)	Meaning
After <s> (near uniform)	two 0.4, one 0.2, book 0.2, what 0.2	1.922	Close to the maximum 2 bits for 4 outcomes: hard to predict the first word

Distribution	Probabilities	H (bits)	Meaning
After "to"	Vellore 0.4, Chennai 0.4, Madurai 0.2	1.522	Moderate: three destinations
After "tickets" (sharply peaked)	to = 1.0	0	No uncertainty: always "to"

Calculation for "to": $-(0.4 \log_2 0.4 + 0.4 \log_2 0.4 + 0.2 \log_2 0.2) = 0.529 + 0.529 + 0.464 = 1.522$. Unigram entropy of the toy corpus is 3.67 bits (maximum $\log_2 16 = 4$). **Meaning:** a peaked distribution makes prediction easy (a model can bet on one word); a flat one makes it hard.

Held-out cross-entropy and perplexity

Toy test set of three utterances (17 tokens including </s>): add-one gives cross-entropy 2.570 bits/word, **perplexity 5.94**; interpolation ($\lambda = 0.7$) gives 1.589 bits/word, **perplexity 3.01**. Per utterance (17 tokens ÷ 3): about 14.6 bits (add-one) and 9.0 bits (interpolation). Fill the comparable table for your held-out set:

Model	Bits/word	Perplexity	Bits/utterance	Utterance perplexity
Unigram	[ ]	[ ]	[ ]	[ ]
Bigram MLE	∞ (zeros)	∞	∞	∞
Bigram add-k	[ ]	[ ]	[ ]	[ ]
Bigram Good–Turing / Katz	[ ]	[ ]	[ ]	[ ]
Trigram Katz	[ ]	[ ]	[ ]	[ ]
Trigram deleted interpolation	[ ]	[ ]	[ ]	[ ]

Per-word and per-utterance figures are different numbers: per-word divides by the number of words, per-utterance by the number of utterances (so it grows with length).

Fair comparison and a deliberately unfair one

Perplexity is comparable only between models that use the **same vocabulary and the same <UNK> treatment**, because it measures the probability of the same event space. **Demonstration:** model A maps every word with count < 5 to <UNK> (vocabulary of 3 in the toy corpus, 57% of tokens become <UNK>); model B keeps all 14 words. Model A's perplexity will look dramatically lower (close to 1–2, since <UNK> follows <UNK> very predictably), but only because it has stopped predicting the words that matter. Compute both on your corpus to show the distortion **[from your run]**.

Rescoring the recogniser hypotheses

Procedure: for each of the 200 hypothesis sets choose the hypothesis maximising acoustic log-score + $w \cdot$ LM log-probability; tune w on the development set; then report WER on the test set before (acoustic best) and after. **Worked example (toy, interpolated LM, similar acoustic scores):**

Hypothesis	$\log_2$ P(LM)	P(LM)	WER vs reference "two tickets to Vellore"
to tickets to Vellore	−12.95	1.3×10^{-4}	25% (1 substitution / 4)
two tickets to Vellore	−5.18	2.8×10^{-2}	0%
two tickets to Velore	−14.17	5.4×10^{-5}	25%

The LM prefers the correct hypothesis by a factor of about 200, so WER drops from 25% to 0% for this set. For all 200 sets report average WER before and after **[from your run]**. **Whether perplexity gains become WER gains:** usually only partly; a lower perplexity shows that the model fits typical text better, but WER only improves when the model separates the specific competing hypotheses (for example proper nouns), so report both and do not assume the link.

Task 6 – English Word Classes and Tagset Design (4 marks)

Open and closed classes

Open classes accept new members easily: nouns, verbs, adjectives, adverbs (new station names appear every month). **Closed classes** have a small fixed membership: prepositions (to, from, at, via), determiners (the, a, this), pronouns (I, you), conjunctions (and, or), modals (can, will, must), auxiliaries (is, do), particles (up, out), wh-words (what, when, which). **Why they matter disproportionately:** closed-class words make up about half of all tokens, are almost never unseen, and are the best cues to the tags of their open-class neighbours (after "the" comes a noun/adjective; after "to" a verb or a place name).

Identification tests for open classes

Class	Morphological test	Distributional test
Noun	takes plural -s, possessive	follows the/a/one: "the ___"
Verb	takes -ing, -ed, 3SG -s	follows to/can/will: "to ___"
Adjective	takes -er, -est, -ly	between determiner and noun: "the late ___ train"
Adverb	often -ly	modifies a verb: "arrives ___"

Ambiguous words from the corpus:

Word	Noun use (test)	Verb use (test)
book	"the book fare" – follows the, takes -s	"book two tickets" – follows please/<s>, takes -ing
return	"a return ticket" – follows a	"return to Chennai" – follows to
stand	"bus stand" – follows noun, takes -s	"please stand" – follows please
fare	"the fare" – follows the	"fare well" – rare, not in domain
change	"the change" – follows the	"change at Katpadi" – follows to/will

Tagset (23 tags) with Penn Treebank mapping

Tag	Description	Example	Penn Treebank
NN	common noun, singular	ticket, fare	NN
NNS	common noun, plural	tickets	NNS
NPL	place name (city, station)	Vellore, Katpadi	NNP, NNPS
NPO	other proper noun (service/train name)	Shatabdi	NNP, NNPS
VB	verb, base/non-3SG present	book, cancel	VB, VBP

Tag	Description	Example	Penn Treebank
VBZ	verb, 3rd-person singular	departs	VBZ
VBD	verb, past tense and participle	cancelled	VBD, VBN
VBG	verb, -ing	booking	VBG
MD	modal	can, will	MD
BE	forms of be/have/do (auxiliary)	is, are	VBZ, VBP, VBD (be/have/do)
JJ	adjective	late, cheap	JJ, JJR, JJS
RB	adverb	soon, tomorrow	RB, RBR, RBS
CD	number, clock time	two, 6:30	CD
DT	determiner	the, a, this	DT, PDT
IN	preposition / subordinating conjunction	from, at, via	IN
TO	infinitive marker "to"	to (book)	TO
CC	coordinating conjunction	and, or	CC
PRP	pronoun	I, my	PRP, PRP$
WH	wh-word	what, when	WDT, WP, WP$, WRB
UH	interjection / politeness word	please, hello	UH
RP	particle	out, off	RP
PUNCT / SYM	punctuation and symbols (₹)	₹, ,	., ,, :, SYM, $

Distinctions made and why: NPL separate from NPO, because extraction needs place names (origin and destination); TO separate from IN, because "to Vellore" (preposition) and "to book" (infinitive) are the main ambiguity; BE separate, because auxiliaries signal tense; CD separate (times, quantities, fares). **Distinctions not made (at least three):** singular/plural proper nouns (NNP/NNPS); comparative/superlative (JJR/JJS, RBR/RBS); VBD vs VBN (past tense vs past participle); VB vs VBP; possessive vs personal pronouns; and the wh-subtypes. Each was dropped because it is rare in short helpline utterances and not needed for extraction.

Annotation and agreement

Report the tag frequency distribution of the 3,000 tokens **[from your run]**. **Raw agreement** = agreed tokens ÷ doubly annotated tokens. **Cohen's kappa** $\kappa = (p_o - p_e) / (1 - p_e)$, where p_o is observed agreement and p_e is agreement expected by chance (from each annotator's tag frequencies). *Illustrative:* 470 of 500 tokens agree, $p_o = 0.94$; $p_e = 0.12$, so $\kappa = (0.94 - 0.12)/(1 - 0.12) =$ **0.93** (above 0.8 is considered strong agreement).

Residual ambiguity

- Proportion of word **types** with more than one tag = (types with ≥2 tags) ÷ (all types); typically about 10–15% of types.
- Proportion of word **tokens** with more than one possible tag = (tokens of ambiguous types) ÷ (all tokens); typically about 35–45% of tokens, since ambiguous words (to, book, return, that) are frequent.
- Average number of tags per ambiguous token (about 2–2.5).

Compute all three on your annotated data **[from your run]**.

Task 7 – Rule-Based Part-of-Speech Tagging (4 marks)

Two-stage architecture

Stage 1: a dictionary assigns every word all the tags it can have (to → {TO, IN}; book → {NN, VB}). **Stage 2:** hand-written constraints remove tags that are impossible in context until one tag remains (the idea of ENGTWOL).

Tag dictionary

Build from the annotated data. Report the number of words with 1, 2, 3 or more candidate tags **[from your run]**. Typical shape: about 85–90% of types have one tag, about 10–15% have two, and only a few have three or more.

Disambiguation rules (condition → action)

#	Condition	Action / context inspected
R1	word = "to" AND previous word ∈ {want, need, going, have, wish}	TO (previous word)
R2	word = "to" AND next tag ∈ {NPL, DT, CD, NNP}	IN (following tag)
R3	cands ∋ {NN, VB} AND previous tag ∈ {DT, JJ, CD}	NN (previous tag)
R4	cands ∋ {NN, VB} AND previous tag ∈ {TO, MD}	VB (previous tag)
R5	cands ∋ {NN, VB} AND word is first in utterance	VB, imperative "Book two tickets" (position)
R6	cands ∋ {NN, VB} AND previous word = "please"	VB (previous word)
R7	cands ∋ {NN, VB} AND next tag ∈ {NN, NNS}, word = "return"	NN, compound "return ticket" (following tag)
R8	word capitalised AND not first AND in station gazetteer	NPL (capitalisation + list)
R9	word capitalised AND not first AND not in gazetteer	NPO (capitalisation)
R10	word ends in -ing AND previous tag = BE	VBG (suffix + previous tag)
R11	word ends in -ed AND cands ∋ VBD	VBD (suffix)
R12	cands ∋ {NNS, VBZ} AND previous tag ∈ {CD, JJ}	NNS (previous tag)
R13	cands ∋ {NNS, VBZ} AND previous tag ∈ {NN, NPL, PRP}	VBZ (previous tag)
R14	word matches \d+(:\d+)? OR next word ∈ {am, pm}	CD (word form / following word)
R15	word ends in -ly	RB (suffix)
R16	unknown word ends in -able/-ive/-al/-ous	JJ (suffix, unknown words)

These use the preceding tag (R3, R4, R10, R12, R13), the following word (R14), the following tag (R2, R7) and the word's own suffix (R10, R11, R15, R16).

Evaluation

Report overall accuracy and accuracy on **ambiguous tokens only** on the held-out set **[from your run]**. The second is the one that matters because unambiguous tokens (about 60% of tokens) are correct by dictionary lookup alone, so they inflate overall accuracy (a baseline that just looks up the dictionary scores highly), while the ambiguous tokens are the only ones the rules decide.

Failure analysis

Five commonest error types (typical for this domain; rank them by your own counts):

- Unknown/new proper nouns not in the gazetteer (tagged NPO or NN instead of NPL).
- NN/VB confusion for book, return, stand in unusual contexts ("return fares apply").
- TO/IN for "to" before a noun used as a verb ("to stand").
- NNS/VBZ for words ending in -s after a noun ("Express stops").
- JJ/NN for noun-modifiers ("bus stand" vs "late bus").

Rule conflict: the utterance "Return tickets cost more" starts with "return": R5 (first word → VB) fires, but R7 (next tag is NNS → NN) also fires. **Resolution:** order specific rules before general ones, so R7 is placed before R5, and the correct tag NN wins. For "Return two tickets", the next tag is CD, R7 does not fire and R5 correctly gives VB. **Cost:** results now depend on rule order; adding any new rule can silently break an older one, every change needs a full regression test, and the rule set is harder to maintain as it grows.

Task 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm (5 marks)

Derivation

Goal: $\hat{t}_1^n = \text{argmax}\ P(t_1^n \mid w_1^n)$. By Bayes' rule: $P(t|w) = P(w|t)P(t) / P(w)$. $P(w)$ is constant for all tag sequences, so $\hat{t} = \text{argmax}\ P(w_1^n \mid t_1^n)\ P(t_1^n)$.

Two independence assumptions: (1) a word depends only on its own tag: $P(w_1^n \mid t_1^n) \approx \Pi\ P(w_i \mid t_i)$ (**emission**); (2) a tag depends only on the previous tag: $P(t_1^n) \approx \Pi\ P(t_i \mid t_{i-1})$ (**transition**, bigram). Result: $\hat{t} = \text{argmax}\ \Pi\ P(w_i|t_i) \cdot P(t_i|t_{i-1})$.

Estimation

Transition: $P(t_i|t_{i-1}) = C(t_{i-1}\ t_i)/C(t_{i-1})$. Emission: $P(w|t) = C(t, w)/C(t)$. Smoothing: add-k (k = 0.01) on transitions (and good practice for emissions on known words) and an open-class prior for unknown words. **Row-sum check (toy tagged corpus of 9 sentences):** row <s>: PRP 3/9 + DT 3/9 + VB 1/9 + UH 1/9 + CD 1/9 = 9/9 = 1; row VB: CD 4/5 + IN 1/5 = 1 (unsmoothed rows sum to 1; after add-k each row still sums to 1 because the k·(T+1) term is in the denominator).

Hand-traced Viterbi: "book two tickets to Vellore"

Toy tagged data (9 sentences) such as "I want to/TO book/VB two/CD tickets/NNS", "book/VB two/CD tickets/NNS to/IN Vellore/NNP", "the/DT book/NN fare/NN to/IN Madurai/NNP". Candidate tags: book {NN, VB}; two {CD}; tickets {NNS}; to {IN, TO}; Vellore {NNP}. Emission: P(book|VB) = 3/5 = 0.6, P(book|NN) = 2/10 = 0.2, P(two|CD) = 2/5 = 0.4, P(tickets|NNS) = 1.0, P(to|IN) = P(to|TO) = 1.0, P(Vellore|NNP) = 2/7 = 0.2857.

Position	Tag	Viterbi value v	Back-pointer	Calculation
1 book	VB	0.0667	<s>	P(VB|<s>)=1/9 × 0.6
1 book	NN	0	<s>	P(NN|<s>)=0 × 0.2 (unsmoothed)
2 two	CD	0.02133	VB	0.0667 × P(CD|VB)=0.8 × 0.4
3 tickets	NNS	0.008533	CD	0.02133 × P(NNS|CD)=0.4 × 1.0
4 to	IN	0.004267	NNS	0.008533 × P(IN|NNS)=0.5 × 1.0
4 to	TO	0	NNS	P(TO|NNS)=0

Position	Tag	Viterbi value v	Back-pointer	Calculation
5 Vellore	NNP	0.001219	IN	0.004267 × P(NNP|IN)=1.0 × 0.2857
End	</s>	0.001219	NNP	× P(</s>|NNP)=1.0

Recovered best sequence (following back-pointers from the end): book/VB two/CD tickets/NNS to/IN Vellore/NNP, **probability 0.001219**. (Unsmoothed, the NN path for "book" is killed by P(NN|<s>) = 0; with add-k smoothing it survives with a tiny value but still loses.)

Comparison: Viterbi, most-frequent-tag baseline, exhaustive search

Method	Accuracy	Operations (this sentence)	General cost
Most-frequent tag	[ ] [from your run]	5 lookups	O(N)
Exhaustive scoring	[ ]	2×1×1×2×1 = 4 sequences × about 11 multiplications = about 44	O(T^N)
Viterbi	[ ]	about 10 transition multiplications (sum of T(i−1)×T(i))	$O(N \cdot T^2)$

With the full tagset (T = 23) and a 5-word utterance with all words fully ambiguous: exhaustive = 23^5 = 6,436,343 sequences; Viterbi ≈ 5×23^2 = 2,645 steps. The baseline is usually about 90% accurate; the HMM adds a few points, mostly on ambiguous tokens **[from your run]**.

Unknown words

An HMM gives P(w|t) = 0 for an unseen word. Fixes: (1) **suffix evidence** – estimate P(suffix | tag) from rare training words, e.g., -ing → VBG, -s → NNS/VBZ, -able → JJ; (2) **capitalisation** – a capitalised word not at the start is very likely a proper noun (NPL/NPO); (3) **open-class prior** – only open-class tags may emit an unknown word, with probability proportional to the number of hapaxes of that tag. Implemented in HMM.B (Appendix) as a capitalisation-based prior; extend it with suffixes. Report accuracy on OOV tokens alone **[from your run]**.

Task 9 – Transformation-Based (Brill) Tagging (4 marks)

Method

- **Initial-state annotator:** tag every word with its most frequent tag (unknown words: NN, or NPL if capitalised).
- **Templates:** patterns such as "change tag A to B if the previous tag is Z".
- **Greedy learning:** in each round instantiate every template with every tag and context, apply each candidate to the training data, and score it as (errors corrected − errors introduced). Select the best transformation, apply it, add it to the ordered list.
- **Stopping condition:** stop when the best score drops below a threshold (for example 2) or the development accuracy stops improving.

Templates (at least six) and window

#	Template: change tag A → B if …	Context
T1	the previous tag is Z	tag at −1

#	Template: change tag A → B if …	Context
T2	the next tag is Z	tag at +1
T3	the previous word is W	word at −1
T4	the next word is W	word at +1
T5	the tag two before is Z	tag at −2
T6	one of the two previous tags is Z	tags −2, −1
T7	the previous tag is Z and the next tag is Y	tags −1, +1
T8	the word is capitalised / ends in suffix S	the word's own form

Window choice: ±2 words/tags. Helpline utterances are short (about 7 words), so a larger window reaches into other clauses and adds noise; most disambiguating cues are the neighbours.

First ten transformations (expected shape; replace with your learner's output)

#	Rule	Corrected	Introduced	Net
1	NN → VB if previous tag is TO	[ ]	[ ]	[ ]
2	IN → TO if next tag is VB	[ ]	[ ]	[ ]
3	VB → NN if previous tag is DT	[ ]	[ ]	[ ]
4	NN → NPL if word is capitalised	[ ]	[ ]	[ ]
5	NN → VB if previous word is "please"	[ ]	[ ]	[ ]
6	NN → VBG if word ends in -ing	[ ]	[ ]	[ ]
7	VBZ → NNS if previous tag is CD	[ ]	[ ]	[ ]
8	TO → IN if next tag is NPL	[ ]	[ ]	[ ]
9	NN → VB if previous tag is MD	[ ]	[ ]	[ ]
10	JJ → NN if next tag is </s>	[ ]	[ ]	[ ]

(The rules are the ones typically learned from this kind of data; the counts must come from your run **[from your run]**.)

Readability, one genuine rule and one artefact

A learned rule is an explicit if-then statement that a human can read, check and edit, whereas HMM probabilities are thousands of numbers whose effect on a decision can only be seen by running the model. **Genuine generalisation:** "NN → VB if the previous tag is TO" (after the infinitive marker comes a verb) holds in English generally. **Artefact:** something like "NNS → NPL if the previous word is 'Chennai'", which fits a few training sentences (for example "Chennai Egmore" patterns) and has no linguistic basis. It will fail on new data.

Accuracy against number of transformations

Apply the first k rules (k = 0, 1, 2, …) and plot the accuracy on **training** and **development** data against k **[from your run]**. The training curve keeps rising, while the development curve rises quickly (first 20–50 rules), flattens, then drops. The **overfitting point** is the k at which the development accuracy peaks; rules beyond it only fit quirks of the training data. Use that k as the stopping point.

Task 10 – Implement and Test the Models (3 marks)

The complete Python module (standard library only; fixed seed 42) is in the Appendix. It contains the vocabulary pipeline, unigram/bigram/trigram counts, MLE, add-k, interpolation, Katz and stupid backoff, Good–Turing, EM for deleted interpolation, entropy/perplexity, WER and rescoring, the rule-based tagger and the HMM with Viterbi. A Brill learner follows the loop in Task 9, with NLTK's `nltk.tag.brill` and `BrillTaggerTrainer` as a ready-made option.

Results table to fill for every model

Model	Training time	Memory	Perplexity / accuracy	Speed (tokens/s)
Bigram add-k	[ ]	[ ]	[ ]	–
Trigram Katz	[ ]	[ ]	[ ]	–
Trigram deleted interpolation	[ ]	[ ]	[ ]	–
Rule-based tagger	–	[ ]	[ ]	[ ]
HMM (Viterbi)	[ ]	[ ]	[ ]	[ ]
Brill tagger	[ ]	[ ]	[ ]	[ ]

Test cases checked against hand calculation (toy corpus; all pass in the supplied code)

Test	Expected (hand-computed)	Code output
Zero probability: MLE P(please|Madurai)	0	0.0
Add-one P(Vellore|to) / P(please|to)	0.1923 / 0.0385	0.1923 / 0.0385
Interpolation P(please|Madurai), λ=0.7	0.00448	0.00448
Katz P(please|Madurai) / P(</s>|Madurai)	0.004386 / 0.75	0.004386 / 0.75
Katz P(please|to) (α = 0.1763)	0.002632	0.002632
Entropy after "to"	1.522 bits	1.522
Perplexity, 3 toy test utterances (add-one / interp.)	5.94 / 3.01	5.94 / 3.01
WER("two tickets to Vellore", "to tickets to Vellore")	0.25	0.25
Viterbi trellis (Task 8)	0.001219, VB-CD-NNS-IN-NNP	run HMM.viterbi on the toy tagged data
λ weights (Task 4)	three bands, as in the Task 4 table	from deleted_interp_lambda on held-out data
First Brill transformations (Task 9)	as in your hand list	compare with the learner's output

Plots to include (at least two)

- Perplexity against n-gram order (unigram, bigram, trigram) for each smoothing method.
- Tagging accuracy against training-set size for the three taggers (learning curves).
- Optional: the Zipf plot (Task 1) and the Brill accuracy-versus-rules curve (Task 9).

Task 11 – Compare, Recommend and Defend the Design (2 marks)

Criterion	Add-k	Katz backoff (Good–Turing)	Deleted interpolation
Perplexity	Highest of the three	Low	Lowest or equal
WER after rescoring	[ ]	[ ]	[ ]
Memory	Small	Large (α tables)	Large (all orders kept)
Speed	Fast	Medium	Medium

Criterion	Rule-based	HMM (Viterbi)	Brill (TBL)
Overall accuracy	Medium	Highest or equal	High
Accuracy on ambiguous tokens	Medium	High	High
Accuracy on unknown tokens	Weak unless rules cover them	Good with suffix/capital prior	Good with lexical templates
Speed	Fast	Medium ($O(N \cdot T^2)$)	Fast
Maintenance	Hardest (rule order)	Needs only data	Readable rules, retrain on new data

(Fill the numeric cells from your measurements **[from your run]**.)

Recommendation: trigram language model with **interpolation (deleted-interpolation weights by context-count band)** or Katz backoff, for rescoring; **HMM tagger** with a suffix and capitalisation model for unknown words, optionally post-corrected by a small Brill rule set. **Why:** the LM gives the lowest development perplexity and a measured WER drop; the HMM gives the best accuracy on ambiguous tokens, needs only annotated data, and handles unknown words probabilistically.

- **Expected accuracy:** [from your measured test-set WER and tagging accuracy].
- **Failure modes:** unseen station names; rare destinations; long-distance dependencies; "to" and "book" in unusual contexts.
- **What a caller experiences:** the system books a ticket to the wrong place or asks the caller to repeat the destination; a mis-tagged destination leads to the wrong fare or timing being read out.

Manual effort: transcription of 20,000 utterances, tagset design, annotation of 3,000 tokens (about 3 hours per 1,000 tokens), double annotation of 500, and rule writing (about 15–30 minutes per rule including testing) must all be costed in hours. **When data beats rules:** once the rule set is large and each new rule corrects very few errors while risking conflicts (the Brill and rule curves flatten), extra annotated data improves an HMM or Brill tagger more per hour than writing more rules; the learning-curve plot in Task 10 shows the point where accuracy is still rising with data but flat with rules.

Task 12 – Day-4 Design Challenge (8 marks)

A. Impact Analysis (2 marks)

What the new input breaks	Effect	Reason
Closed vocabulary	More than 15% of tokens become <UNK>	Hundreds of new station and train names per month
Stable unknown-word rate	OOV rate no longer constant (about 3–4% → more than 15%)	Vocabulary grows every month; the model goes out of date

What the new input breaks	Effect	Reason
English-only word classes	Tags and rules assume English	Code-mixed words (enna, venum) have no English tag
Transition probabilities from a narrow domain	Bus-only transitions are wrong for rail/metro language	New utterance patterns (platform, coach, class)
Tagset	No category for code-mixed tokens	Needs a new tag CM (or language-tagged categories)

Why the OOV words are the worst to lose: they are mostly the place names, which are exactly what the caller's request turns on (origin, destination). An <UNK> carries no identity, so the LM cannot choose between competing hypotheses that differ in the place name, and the tagger has no cue to mark it NPL. Function words are never OOV, so the loss falls on the most informative words.

Estimated effect (to be checked by measurement): tagging accuracy ≈ (1 − u) · A_known + u · A_unknown. With A_known = 96%, A_unknown = 80% (from Task 8): baseline u = 3% → 95.5%; new u = 15% → 93.6%; if code-mixed OOV tokens are only tagged at 50%: 0.85×96 + 0.15×50 = 89%. For perplexity, 15% UNK tokens (instead of 3%) and context counts estimated on a different domain raise cross-entropy by roughly 0.5–1 bit, so a perplexity of 90 would rise to about 130–180 *(estimate)*.

B. Evaluation of the Original Design (2 marks)

Utterance	Trace	What fails first	Why
Unseen station: "two tickets to Katpadi"	"Katpadi" → <UNK>; LM P(<UNK>|to) is a small, shared value; HMM tags by capital-letter prior → NPL (correct but guessed)	LM: all place names are treated as the same <UNK>	closed vocabulary
Code-mixed: "Chennai to Madurai bus timing enna"	"enna" → <UNK>; LM sees <UNK> after "timing"; HMM tags it NN	Tagger: "enna" (what) should be WH	English-only classes
Both: "Katpadi ku ticket venum"	"Katpadi" <UNK>, "ku" <UNK>, "venum" <UNK>; HMM gives each NN/NPL	LM: three <UNK> in a row; extraction: no destination identified	closed vocab + non-English

Measure on the noisy sample and report against the original baselines: perplexity, WER after rescoring, tagging accuracy (overall and on OOV tokens) **[from your run]**. **Component that degrades fastest:** the language model (perplexity and rescoring), because place names disappear into <UNK>; the tagger degrades less because suffix/capital evidence still works. **More economical approaches:** a **gazetteer** of station names (cheap, updated monthly without retraining); a **class-based n-gram** (replace names with <STATION> so a new name inherits the class statistics); a **character-level model** for spelling variants and phonetic transcription of code-mixed words; and an **open-vocabulary tagger** that uses character features.

C. Adaptation or Redesign (2 marks)

Decision: MODIFY (keep the n-gram LM and HMM tagger structure, add the missing parts).

Component	Change
Vocabulary	Replace named entities by class tokens (<STATION>, <TRAIN>, <FARE_CLASS>) using a gazetteer; keep <UNK> only for truly unknown words; add a code-mixed lexicon (enna, venum, ku)
Smoothing	Class-based n-gram plus lower-order backoff; add a character n-gram model for P(<UNK> word spelling)

Component	Change
Backoff weights	Re-estimate the λ weights by deleted interpolation on a held-out set that includes rail and code-mixed calls
Tagset	Add tag CM for code-mixed tokens (24 tags), with a mapping such as CM → FW in the Penn Treebank mapping
Tagger	HMM with character-suffix and gazetteer emission for unknown words; retrain on annotated rail and code-mixed utterances; token-level language ID before tagging

Effect (estimates to be replaced by measurement): with a gazetteer, the effective OOV rate on place names falls from about 15% to about 3–5%, perplexity returns to near the original level, and tagging accuracy recovers to about 94–95% on the new sample. **Retraining frequency:** gazetteer update weekly or monthly (no retraining, only a list edit); full retraining of LM and tagger quarterly, or when the OOV rate on the latest calls exceeds a threshold (for example 8%). **Annotation effort:** extra tagged rail/code-mixed data (about 2,000–3,000 tokens) and gazetteer maintenance are the continuing cost.

D. Evidence-Based Defence (2 marks)

- **Perplexity tables and rescoring (Task 5):** the n-gram model reduced WER on the clean data, so rescoring itself is the right approach; the new problem is vocabulary, not the method.
- **Tagging accuracy and error analysis (Tasks 7–9):** most unknown-word errors are proper nouns and code-mixed words, which a gazetteer, a CM tag and suffix evidence directly address.
- **OOV measurement (Task 1):** the 15% rate shows that more data alone cannot fix the problem, since new names arrive every month; classes and a gazetteer scale better.
- **Effort estimates (Task 11):** a full redesign (for example a neural model) would discard the existing resources and need far more annotated data; modification costs less and keeps everything that already works.

Remaining limitations: an n-gram model can never capture long-distance dependencies (for example agreement between the first and last parts of a long utterance), meaning or intent beyond a window of one or two words, or true semantic ambiguity, however much data it is given; it also cannot generalise from a name it has never seen unless a class or gazetteer exists. A learned neural model with sub-word units is the natural next step once enough annotated data exists.

Appendix – Python Code (standard library only)

File: lm_pos.py (also supplied as a separate file). It reproduces every toy-corpus figure in this document.

```
"""N-gram LM + POS taggers for the transport helpline (stdlib only; NLTK optional)."""
import math, random
from collections import Counter, defaultdict


random.seed(42)                          # fixed seed for every experiment

# ---------- Task 1: counting and vocabulary ----------
def tokenise(line):
    return line.lower().split()          # case folded; "6:30" kept as one token

def build_vocab(sents, min_count=2):
    c = Counter(w for s in sents for w in s)
    return {w for w, n in c.items() if n >= min_count}

def mark(sents, vocab):
    return [["<s>"] + [w if w in vocab else "<UNK>" for w in s] + ["</s>"] for s in sents]

# ---------- Tasks 2-4: models ----------
class NGram:
```

```
    def __init__(self, sents):
        self.u, self.b, self.t = Counter(), Counter(), Counter()
        for s in sents:
            self.u.update(s)
            self.b.update(zip(s, s[1:]))
            self.t.update(zip(s, s[1:], s[2:]))
        self.N, self.V = sum(self.u.values()), len(self.u)
        self.ctx2 = Counter()
        for (a, b, c), n in self.t.items():
            self.ctx2[(a, b)] += n

    def mle_bi(self, a, b):
        return self.b[(a, b)] / self.u[a] if self.u[a] else 0.0

    def mle_tri(self, a, b, c):
        return self.t[(a, b, c)] / self.ctx2[(a, b)] if self.ctx2[(a, b)] else 0.0

    def addk(self, a, b, k=1.0):                # add-one when k = 1
        return (self.b[(a, b)] + k) / (self.u[a] + k * self.V)

    def interp(self, a, b, lam=0.7):            # bigram + unigram
        return lam * self.mle_bi(a, b) + (1 - lam) * self.u[b] / self.N

    def interp3(self, a, b, c, lams=(0.6, 0.3, 0.1)):
        l3, l2, l1 = lams
        return l3 * self.mle_tri(a, b, c) + l2 * self.mle_bi(b, c) + l1 * self.u[c] / self.N

    def katz_bi(self, a, b, d=0.5):             # absolute discount d stands in for Good-Turing
        if self.b[(a, b)] > 0:
            return (self.b[(a, b)] - d) / self.u[a]
        seen = [w for (x, w) in self.b if x == a]
        beta = 1 - sum((self.b[(a, w)] - d) / self.u[a] for w in seen)
        denom = 1 - sum(self.u[w] / self.N for w in seen)
        return beta / denom * self.u[b] / self.N  # alpha * P_uni

    def stupid_bi(self, a, b, lam=0.4):         # a score, not a probability
        return self.b[(a, b)] / self.u[a] if self.b[(a, b)] else lam * self.u[b] / self.N

def good_turing(counter):
    Nr = Counter(counter.values())
    star = {c: (c + 1) * Nr[c + 1] / Nr[c] for c in Nr if Nr[c + 1]}
    p_unseen_total = Nr[1] / sum(counter.values())
    return Nr, star, p_unseen_total

def deleted_interp_lambda(heldout_pairs, model, iters=20):
    """EM for two weights (bigram, unigram) on held-out data."""
    l2 = 0.5
    for _ in range(iters):
        e2 = e1 = 0.0
        for a, b in heldout_pairs:
            p2, p1 = l2 * model.mle_bi(a, b), (1 - l2) * model.u[b] / model.N
            z = p2 + p1
            if z: e2 += p2 / z; e1 += p1 / z
        l2 = e2 / (e2 + e1)
    return l2, 1 - l2

# ---------- Task 5: entropy, perplexity ----------
def entropy(dist):
    return -sum(p * math.log2(p) for p in dist if p > 0)

def perplexity(sents, prob):
    lp, n = 0.0, 0
    for s in sents:
        for a, b in zip(s, s[1:]):
            p = prob(a, b)
            lp += math.log2(p) if p > 0 else float("-inf"); n += 1
    return 2 ** (-lp / n), -lp / n              # (perplexity, cross-entropy bits/word)

def wer(ref, hyp):
    r, h = ref.split(), hyp.split()
    d = [[0] * (len(h) + 1) for _ in range(len(r) + 1)]
```

```
    for i in range(len(r) + 1): d[i][0] = i
    for j in range(len(h) + 1): d[0][j] = j
    for i in range(1, len(r) + 1):
        for j in range(1, len(h) + 1):
            d[i][j] = min(d[i-1][j] + 1, d[i][j-1] + 1, d[i-1][j-1] + (r[i-1] != h[j-1]))
    return d[-1][-1] / len(r)

def rescore(hyps, lm_logprob, acoustic, weight=1.0):
    """hyps: list of strings; acoustic: list of log-scores. Returns best hypothesis."""
    return max(zip(hyps, acoustic), key=lambda x: x[1] + weight * lm_logprob(x[0]))[0]

# ---------- Task 7: rule-based tagger ----------
def rule_tag(words, tagdict):
    tags = []
    for i, w in enumerate(words):
        cands = list(tagdict.get(w.lower(), ["NN"]))
        prev = tags[i-1] if i else "<s>"
        nxt = words[i+1].lower() if i + 1 < len(words) else "</s>"
        if w.lower() == "to":                                   # R1/R2
            cands = ["TO"] if prev in ("VB", "MD") or (i and words[i-1].lower() in {"want", "need"}) else ["IN"]
        elif set(cands) >= {"NN", "VB"}:
            cands = ["NN"] if prev in ("DT", "JJ", "CD") else ["VB"] if prev in ("TO", "MD", "<s>") else cands
        tags.append(cands[0])
    return tags

# ---------- Task 8: HMM + Viterbi ----------
class HMM:
    def __init__(self, tagged, k=0.01):
        self.tr, self.em, self.tc = Counter(), Counter(), Counter()
        self.k = k
        for s in tagged:
            t = ["<s>"] + [x[1] for x in s] + ["</s>"]
            self.tr.update(zip(t, t[1:])); self.tc.update(t[:-1])
            for w, tag in s: self.em[(tag, w)] += 1
        self.tags = sorted({tag for (tag, _) in self.em})
        self.ec = Counter()
        for (tag, _), n in self.em.items(): self.ec[tag] += n

    def A(self, a, b): return (self.tr[(a, b)] + self.k) / (self.tc[a] + self.k * (len(self.tags) + 1))
    def B(self, t, w):
        if self.em[(t, w)]: return self.em[(t, w)] / self.ec[t]
        return 1e-6 * (1.0 if w[:1].isupper() and t in ("NPL", "NNP") else 0.1)   # unknown-word prior

    def viterbi(self, words):
        V = [{t: self.A("<s>", t) * self.B(t, words[0]) for t in self.tags}]
        bp = [{}]
        for w in words[1:]:
            col, ptr = {}, {}
            for t in self.tags:
                best = max(self.tags, key=lambda p: V[-1][p] * self.A(p, t))
                col[t] = V[-1][best] * self.A(best, t) * self.B(t, w); ptr[t] = best
            V.append(col); bp.append(ptr)
        last = max(self.tags, key=lambda t: V[-1][t] * self.A(t, "</s>"))
        path = [last]
        for i in range(len(words) - 1, 0, -1): path.append(bp[i][path[-1]])
        return list(reversed(path)), V
```